Lecture 5: Neural Networks

Part 1: From Image Features to Neural Networks

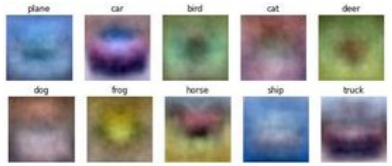
Where we are:

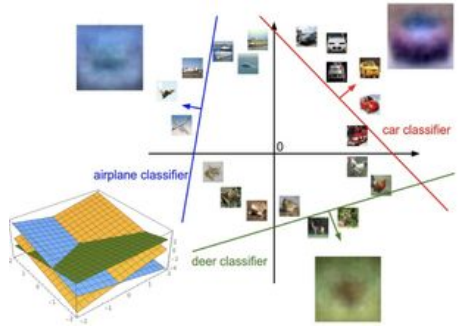
1. Linear models map images to class scores.

2. Loss functions measure whether the scores match the labels.

3. Stochastic gradient descent updates the weights to reduce the loss.

$$s = f(x; W) = Wx$$



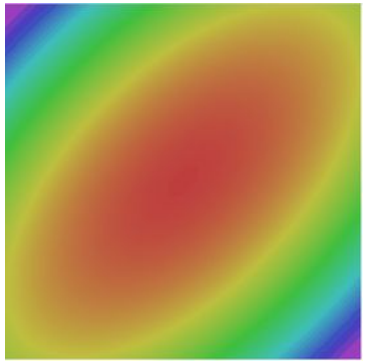


$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$ Softmax

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ SVM

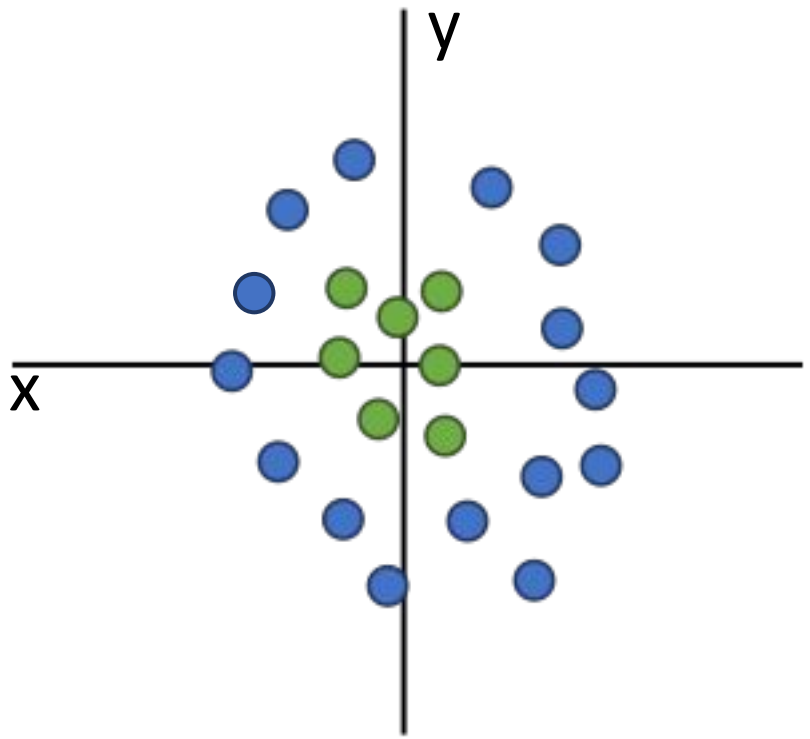
$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + R(W)$$

```
v = 0
for t in range(num_steps):
  dw = compute_gradient(w)
  v = rho * v + dw
  w -= learning_rate * v
```



Problem: Linear Classifiers aren't that powerful

Geometric Viewpoint



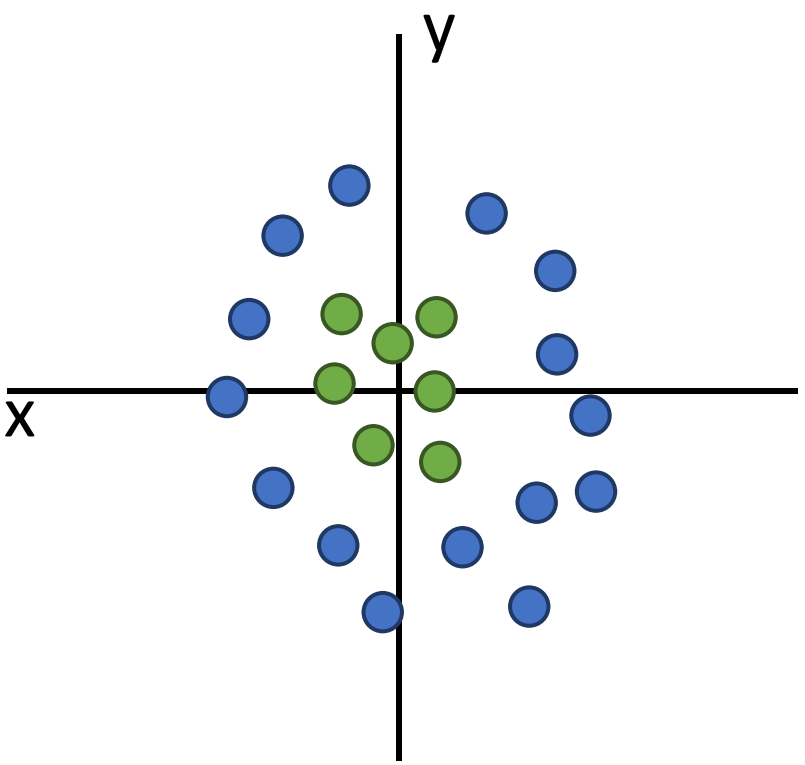
Visual Viewpoint

One template per class:
Can't recognize different
modes of a class



One solution: Feature Transforms

Original space

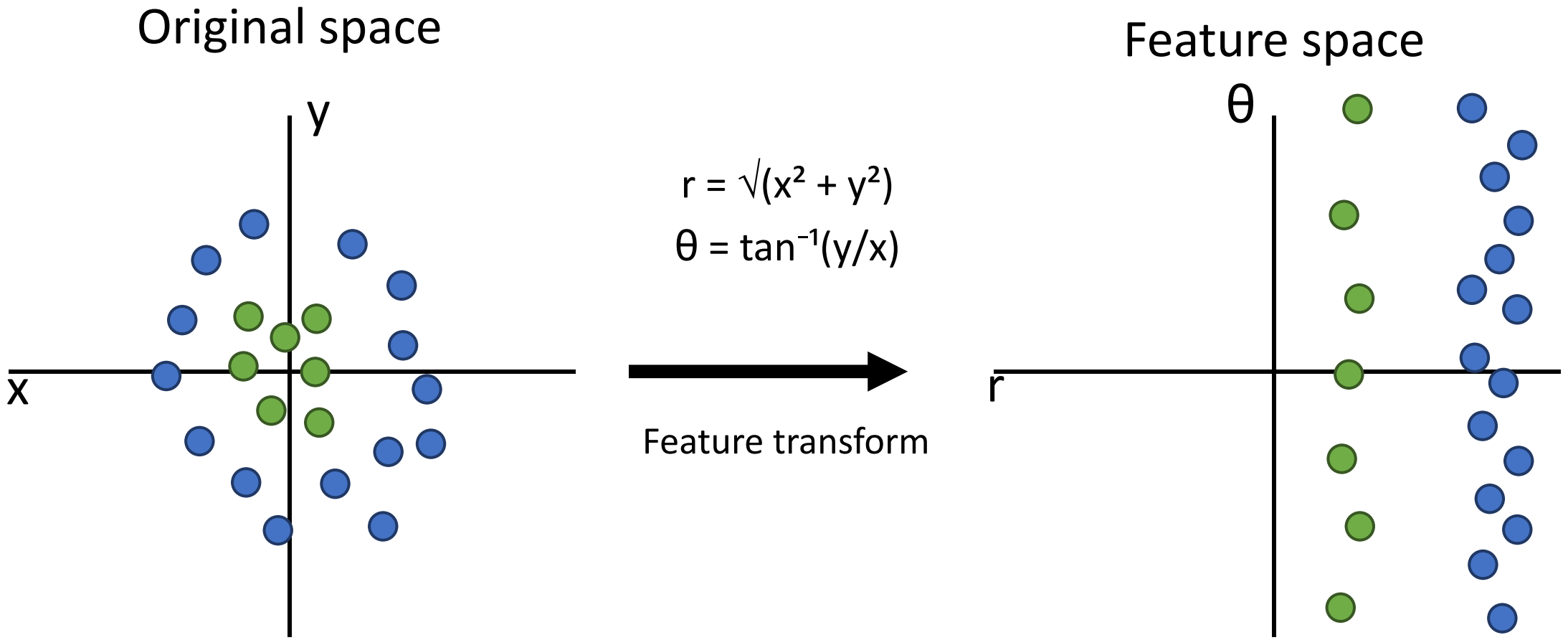


$r = \sqrt{(x^2 + y^2)}$

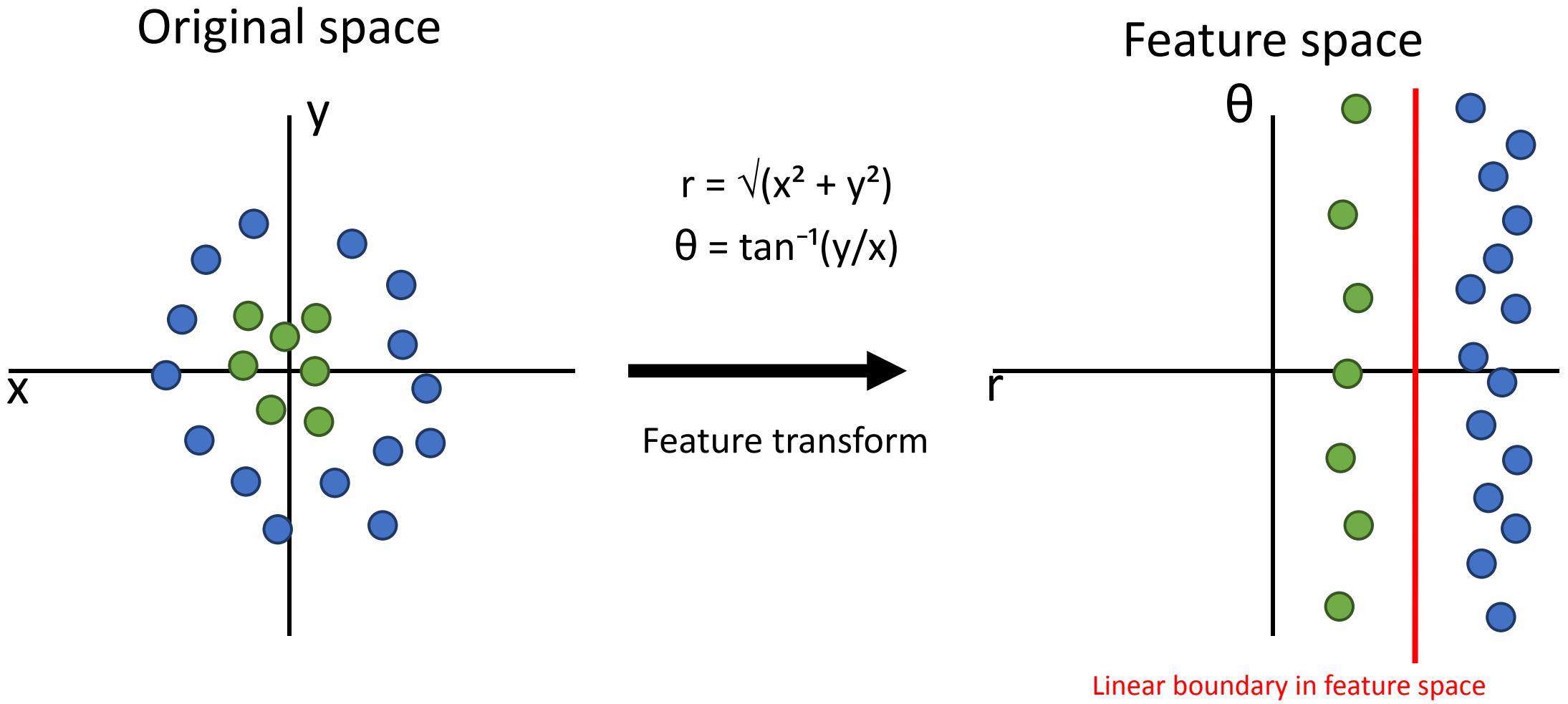
$\theta = \tan^{-1}(y/x)$

Feature transform

One solution: Feature Transforms



One solution: Feature Transforms



One solution: Feature Transforms

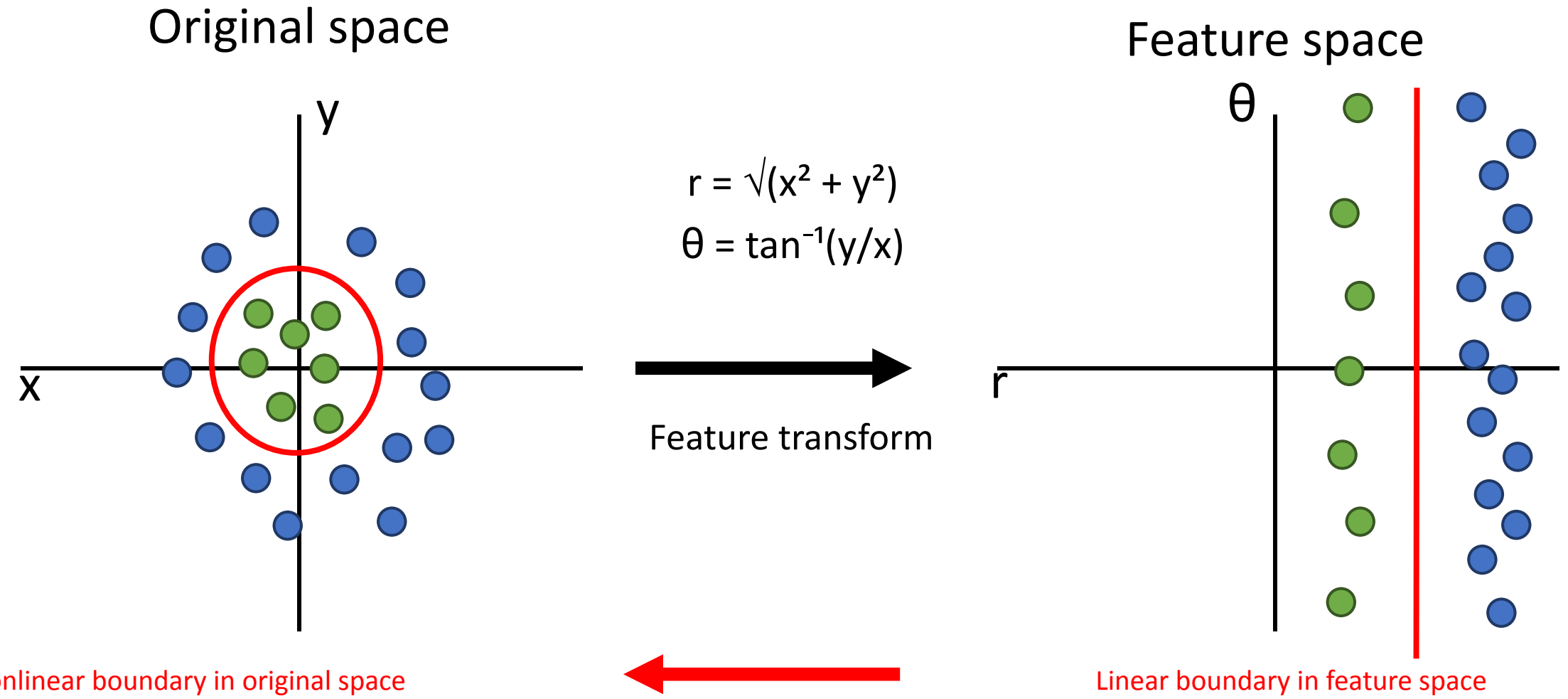
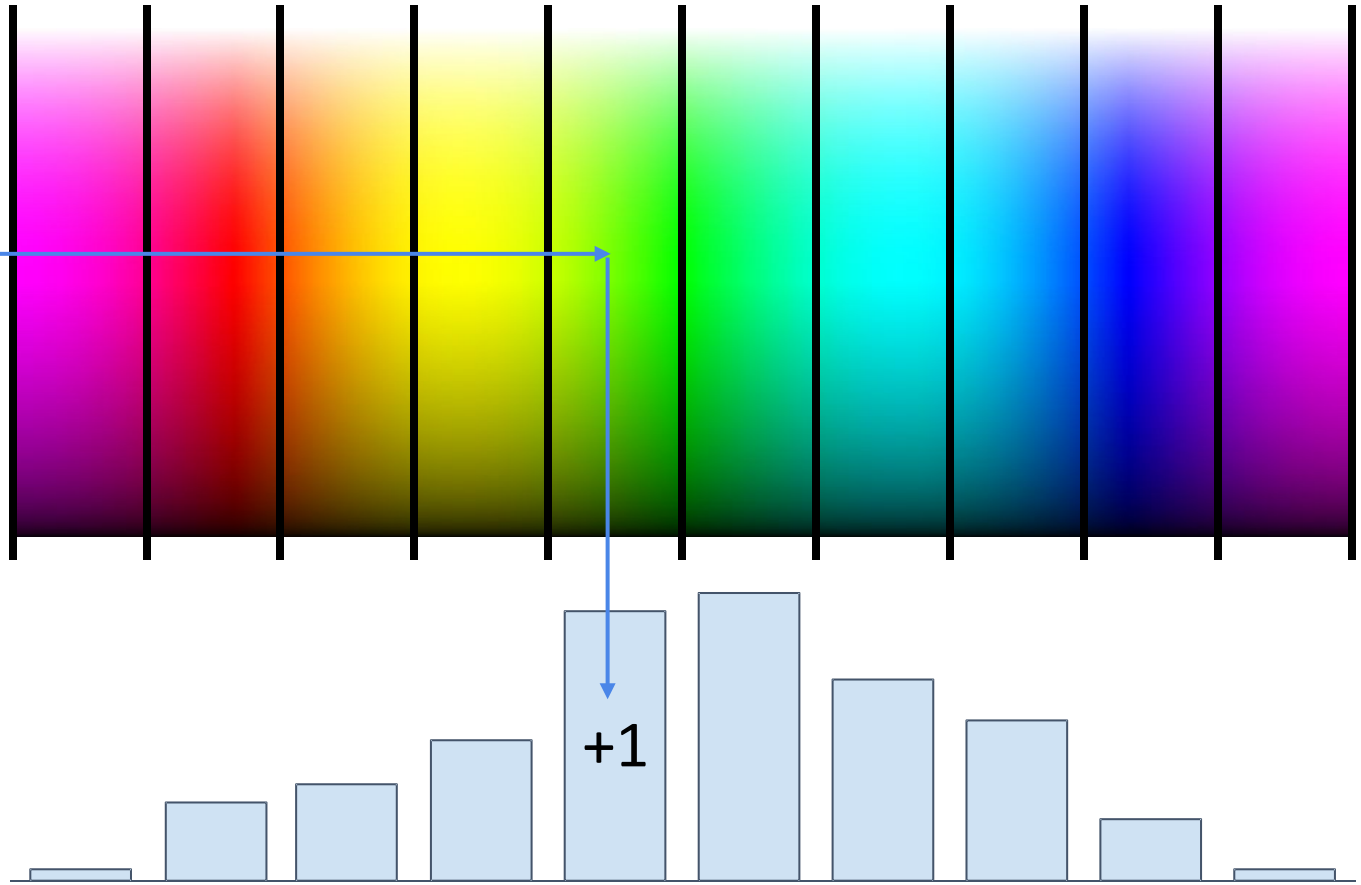


Image Features: Color Histogram





Ignores texture,

spatial positions

Image Features: Histogram of Oriented Gradients (HoG)



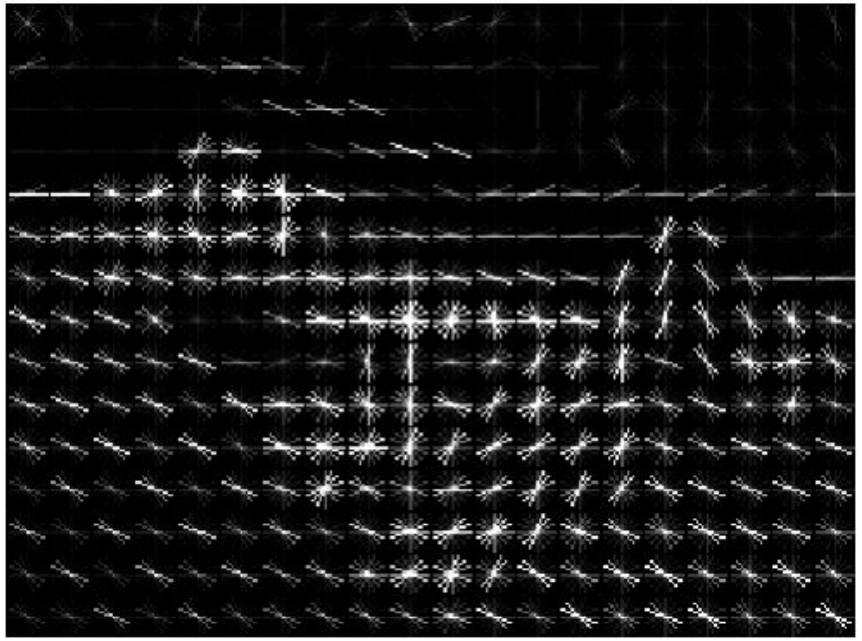
1. Compute pixel gradient direction and magnitude.
2. Divide the image into 8 × 8 pixel cells.
3. Form a weighted orientation histogram per cell.

Lowe, “Object recognition from local scale-invariant features”, ICCV 1999 Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

Image Features: Histogram of Oriented Gradients (HoG)





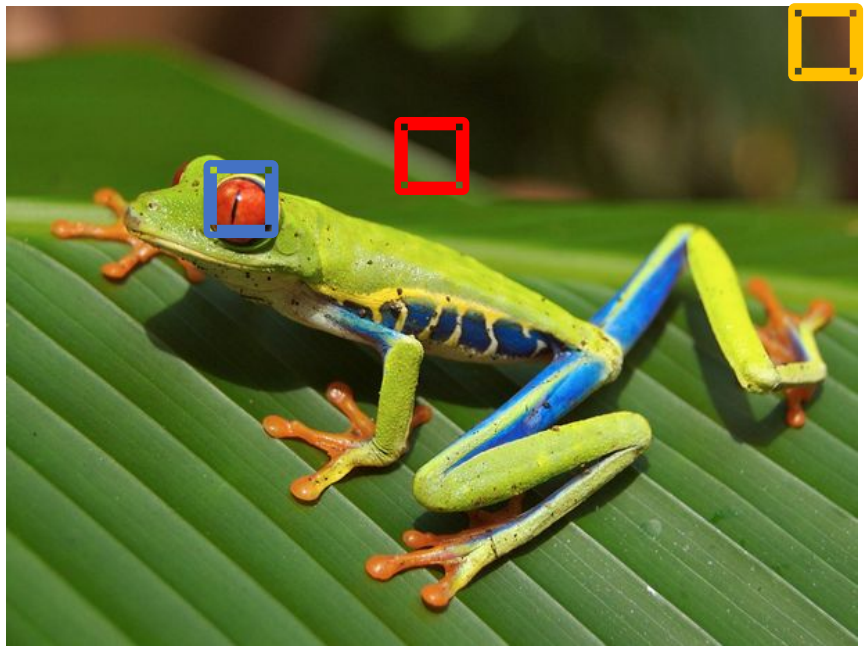


1. Compute pixel gradient direction and magnitude.
2. Divide the image into 8 × 8 pixel cells.
3. Form a weighted orientation histogram per cell.

Example: A 320 × 240 image produces 40 × 30 cells. With 9 orientation bins per cell, the feature vector contains 40 × 30 × 9 = 10,800 values.

Lowe, “Object recognition from local scale-invariant features”, ICCV 1999 Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

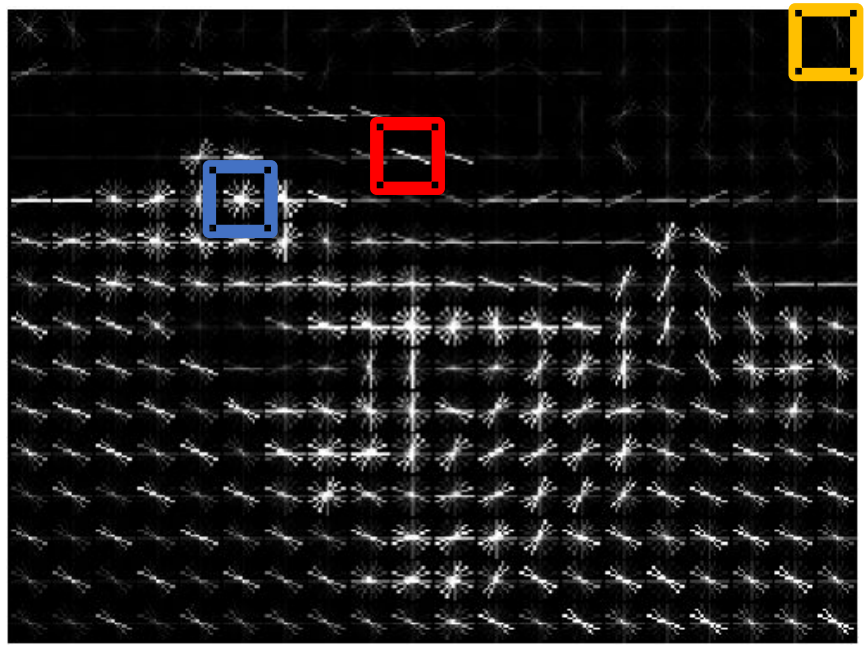
Image Features: Histogram of Oriented Gradients (HoG)



Weak edges

Strong diagonal edges

Edges in all directions

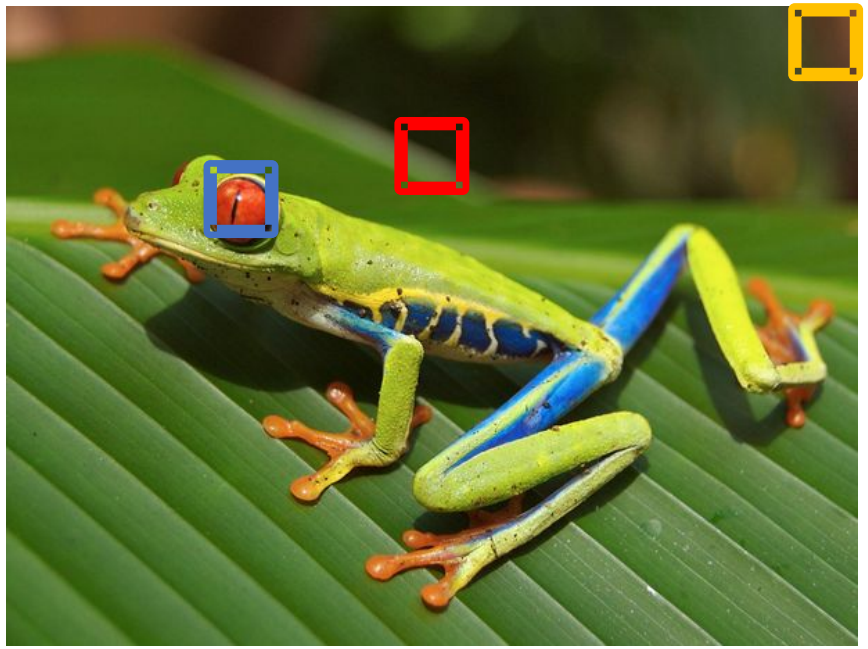


1. Compute pixel gradient direction and magnitude.
2. Divide the image into 8 × 8 pixel cells.
3. Form a weighted orientation histogram per cell.

Example: A 320 × 240 image produces 40 × 30 cells. With 9 orientation bins per cell, the feature vector contains 40 × 30 × 9 = 10,800 values.

Lowe, “Object recognition from local scale-invariant features”, ICCV 1999 Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

Image Features: Histogram of Oriented Gradients (HoG)



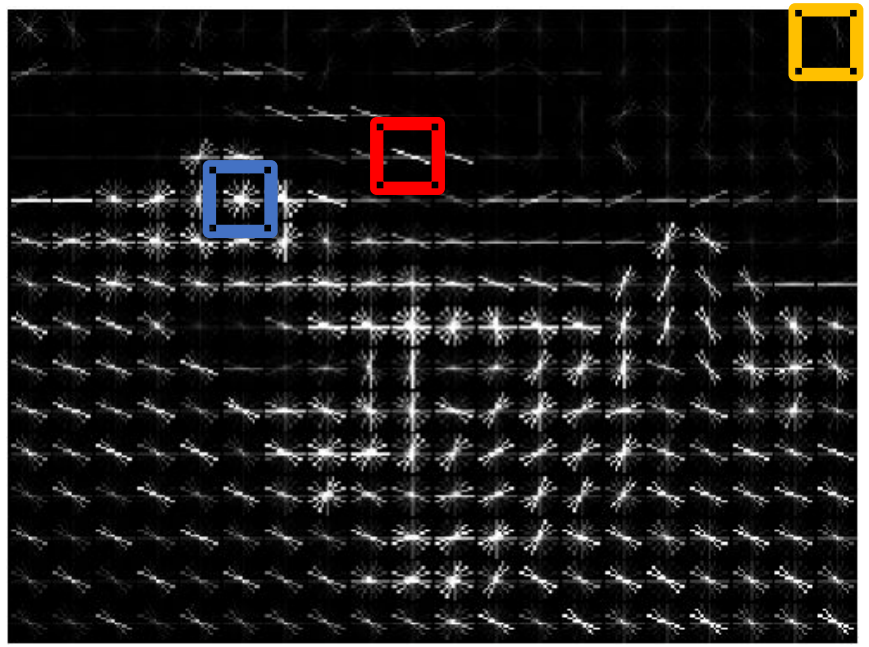
1. Compute pixel gradient direction and magnitude.
2. Divide the image into 8 × 8 pixel cells.
3. Form a weighted orientation histogram per cell.

Weak edges

Strong diagonal edges

Edges in all directions

Captures texture and position, robust to small image changes



Example: A 320 × 240 image produces 40 × 30 cells. With 9 orientation bins per cell, the feature vector contains 40 × 30 × 9 = 10,800 values.

Lowe, “Object recognition from local scale-invariant features”, ICCV 1999 Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

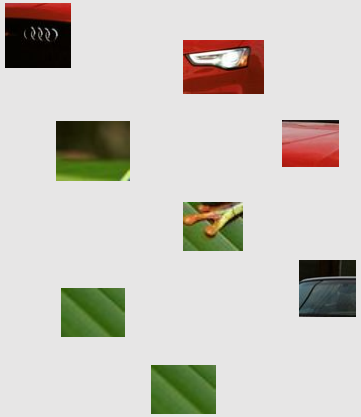
Image Features: Bag of Words (Data-Driven!)

Step 1: Build codebook





Extract random patches



Cluster patches to form “codebook” of “visual words”



Fei-Fei and Perona, “A bayesian hierarchical model for learning natural scene categories”, CVPR 2005

Car image is CC0 1.0 public domain

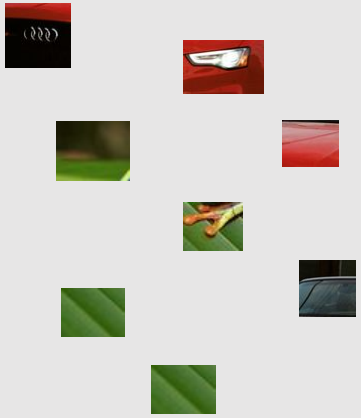
Image Features: Bag of Words (Data-Driven!)

Step 1: Build codebook





Extract random patches

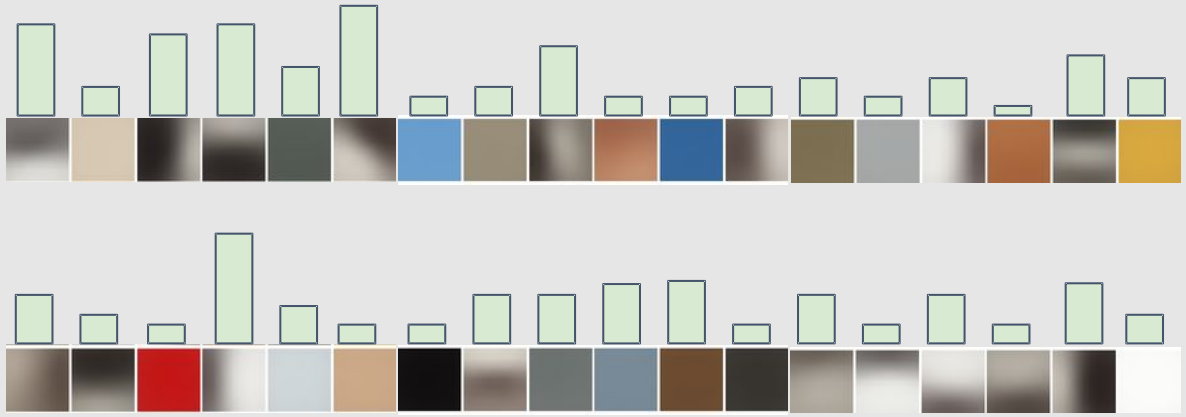


Cluster patches to form “codebook” of “visual words”



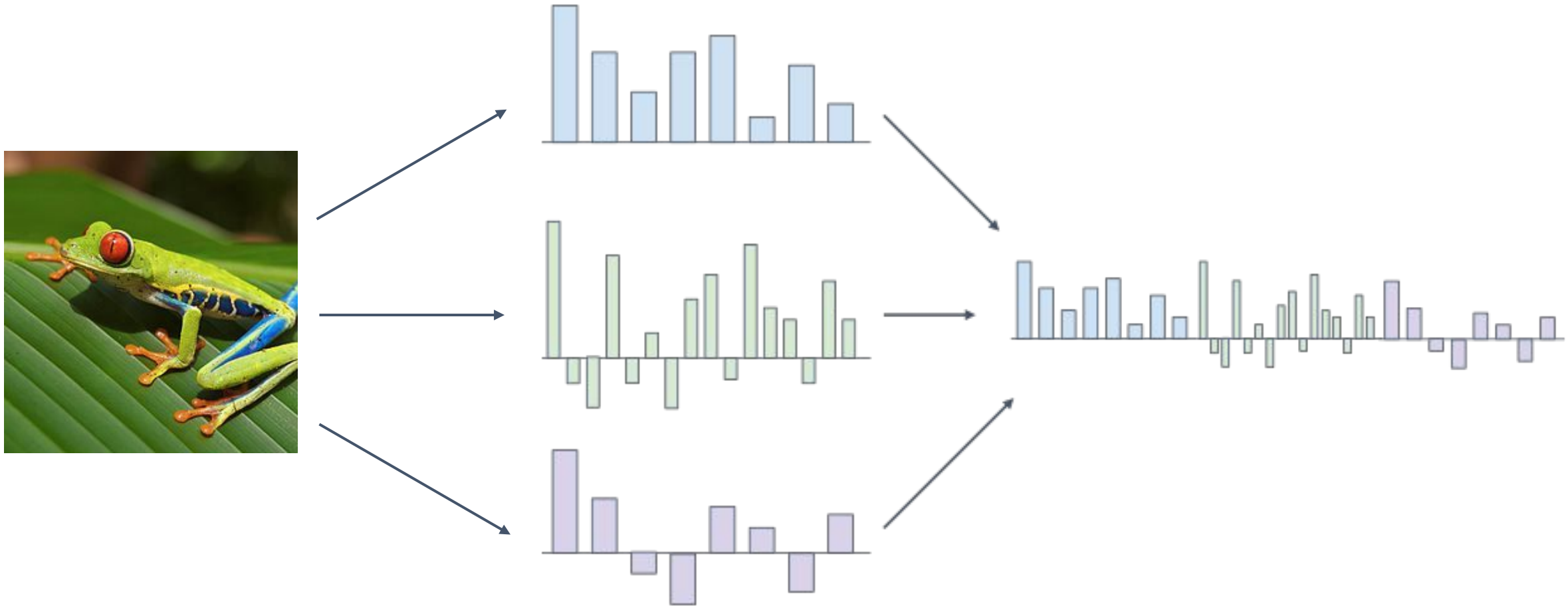
Step 2: Encode images





Fei-Fei and Perona, “A bayesian hierarchical model for learning natural scene categories”, CVPR 2005

Image Features



Example: Winner of 2011 ImageNet challenge

Low-level feature extraction ≈ 10k patches per image

- SIFT: 128-dim
- color: 96-dim

} reduced to 64-dim with PCA

FV extraction and compression:

- N=1,024 Gaussians, R=4 regions ⇨ 520K dim x 2
- compression: G=8, b=1 bit per dimension

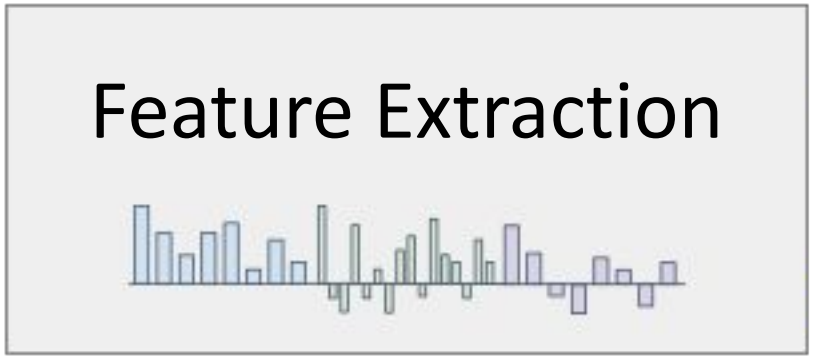
One-vs-all SVM learning with SGD

Late fusion of SIFT and color systems

F. Perronnin, J. Sánchez, "Compressed Fisher vectors for LSVRC", PASCAL VOC / ImageNet workshop, ICCV, 2011.

Image Features





f



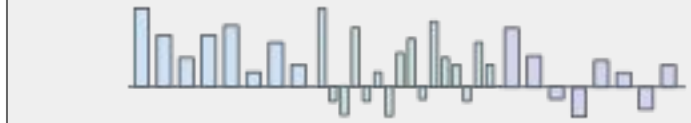
10 numbers giving scores for classes



training

Image Features vs Neural Networks



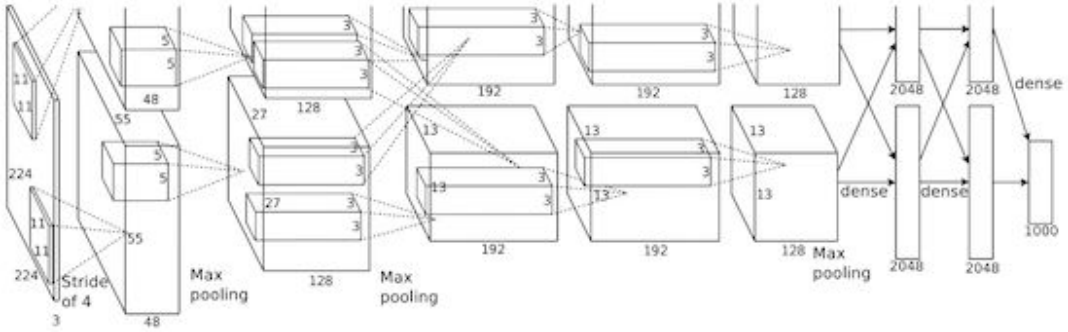


f

One score per class

training





Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012. Figure copyright Krizhevsky, Sutskever, and Hinton, 2012. Reproduced with permission.

One score per class

training

Neural Networks

(**Before**) Linear score function:

$$f = Wx$$

$$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$$

Neural Networks

(**Before**) Linear score function:

$$f = Wx$$

(**Now**) 2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

$$W_2 \in \mathbb{R}^{C \times H} \quad W_1 \in \mathbb{R}^{H \times D} \quad x \in \mathbb{R}^D$$

(In practice we will usually add a learnable bias at each layer as well)

Neural Networks

(**Before**) Linear score function:

$$f = Wx$$

(**Now**) 2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

$$W_3 \in \mathbb{R}^{C \times H_2} \quad W_2 \in \mathbb{R}^{H_2 \times H_1} \quad W_1 \in \mathbb{R}^{H_1 \times D} \quad x \in \mathbb{R}^{D}$$

(In practice we will usually add a learnable bias at each layer as well)

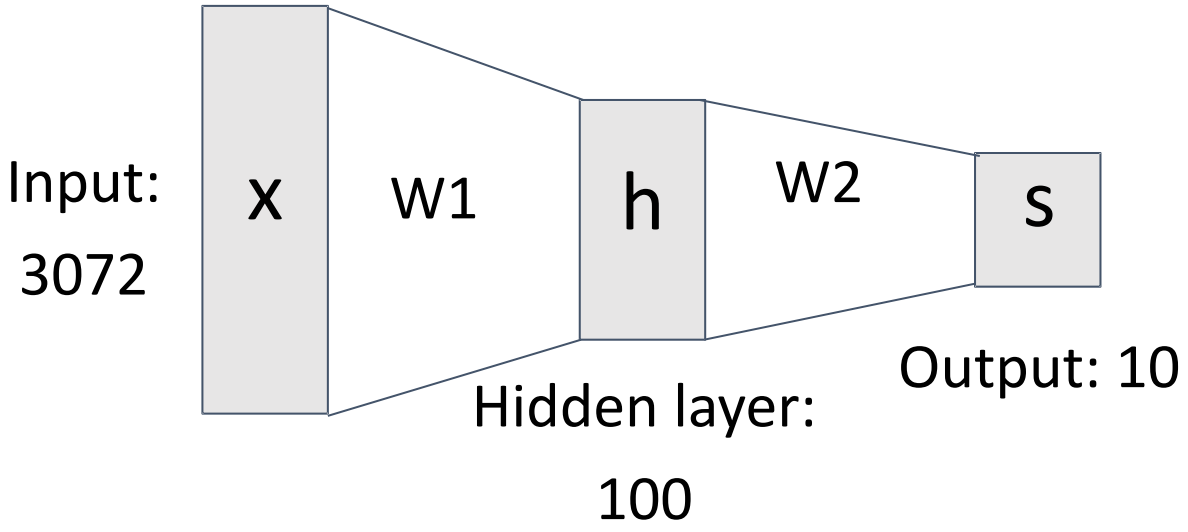
Neural Networks

(**Before**) Linear score function:

$$f = Wx$$

(**Now**) 2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural Networks

(**Before**) Linear score function:

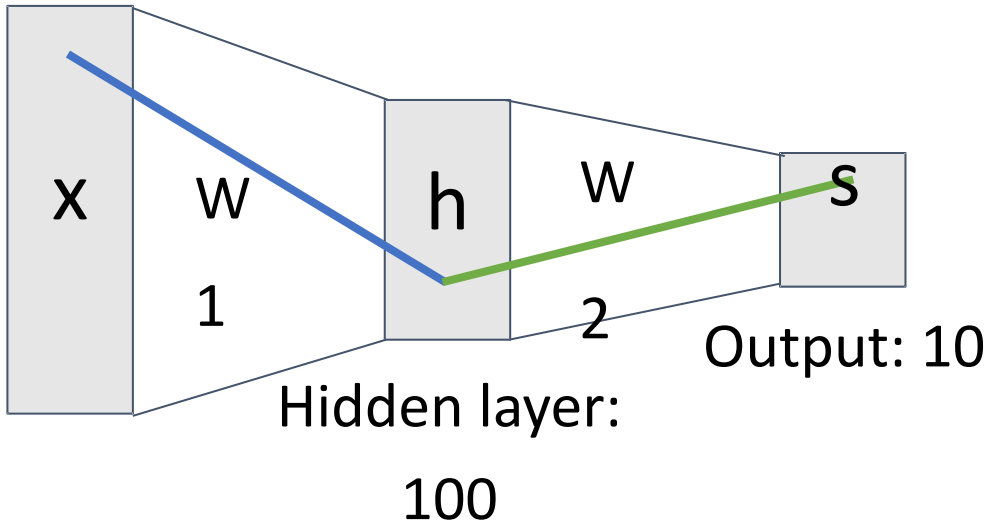
$$f = Wx$$

(**Now**) 2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

Element (i, j)

of W1 gives
the effect on
h_i from x_j



Element (i, j)

of W2 gives
the effect on
S_i from h_j

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural Networks

(**Before**) Linear score function:

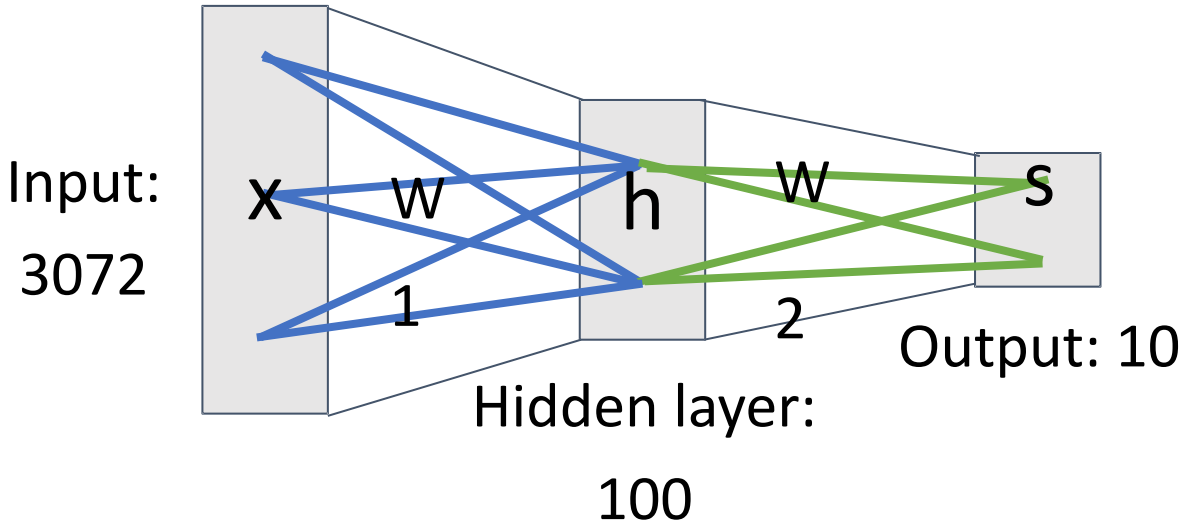
$$f = Wx$$

(**Now**) 2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

Element (i, j) of W1 gives the effect on hi from xj

All elements of x affect all elements of h



Element (i, j) of W2 gives the effect on si from hj

All elements of h affect all elements of s

Fully-connected neural network

Also “Multi-Layer Perceptron” (MLP)

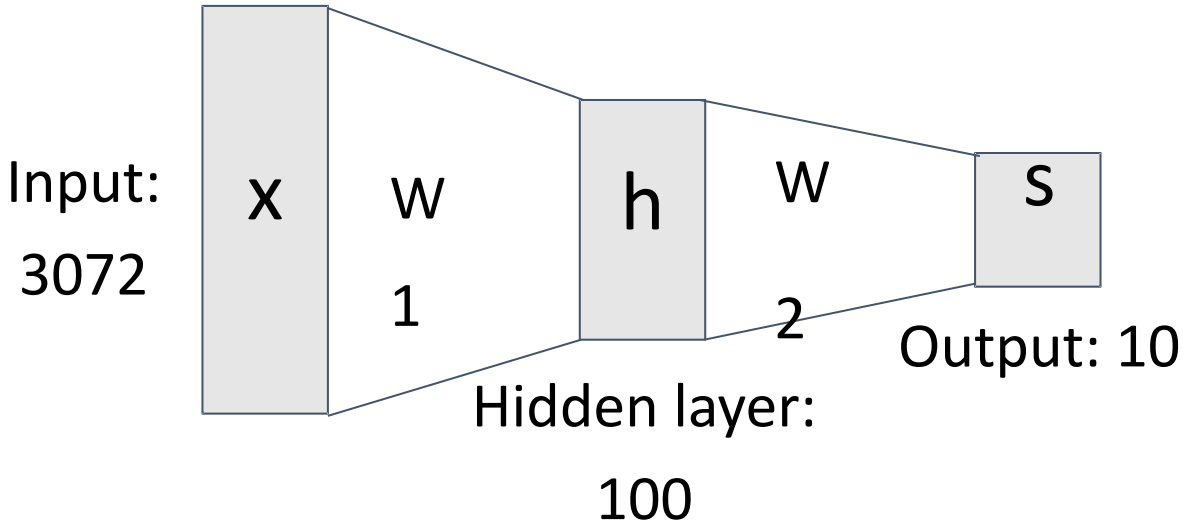
Neural Networks

Linear classifier: One template per class



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural Networks

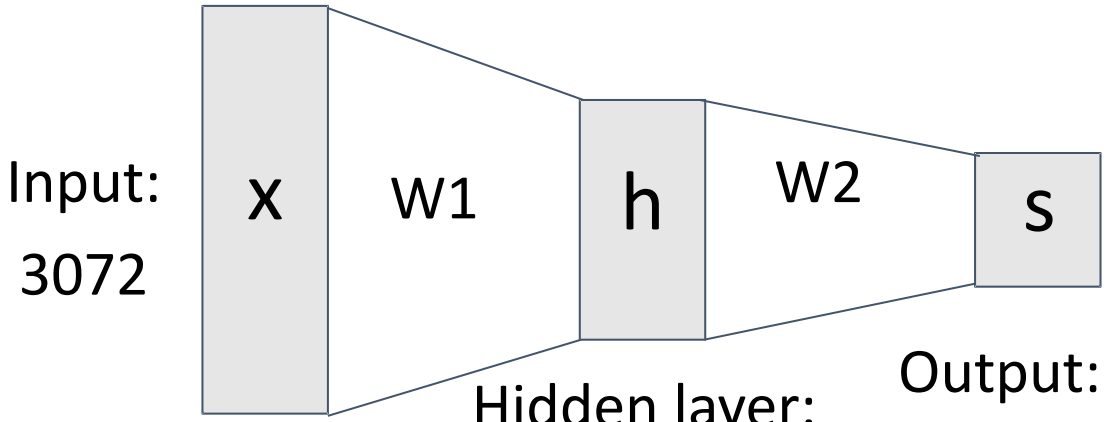
Neural net: first layer is bank of templates;

Second layer recombines templates



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

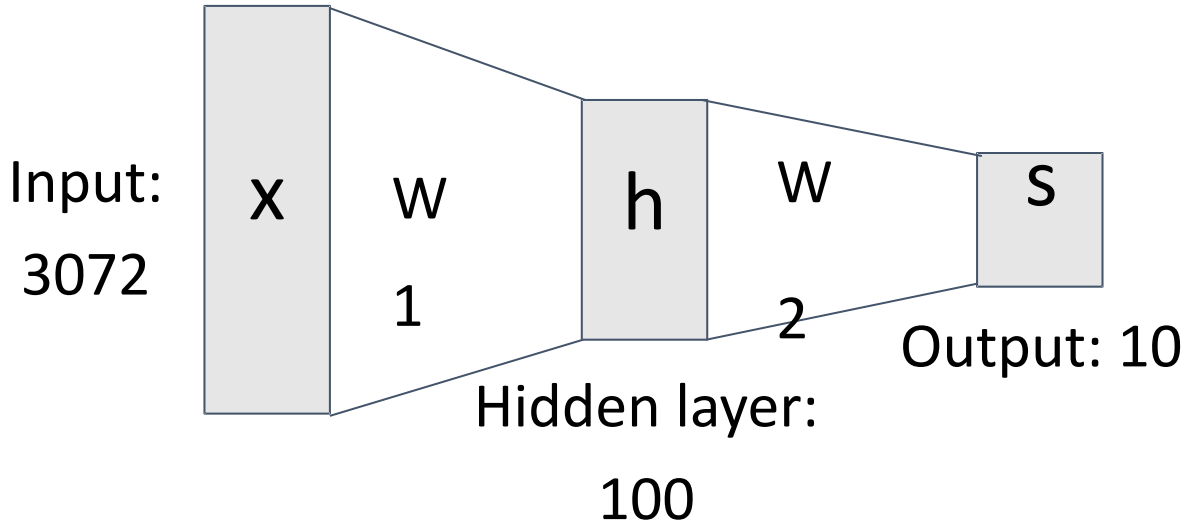
Neural Networks

Can use different templates to cover multiple modes of a class!



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural Networks

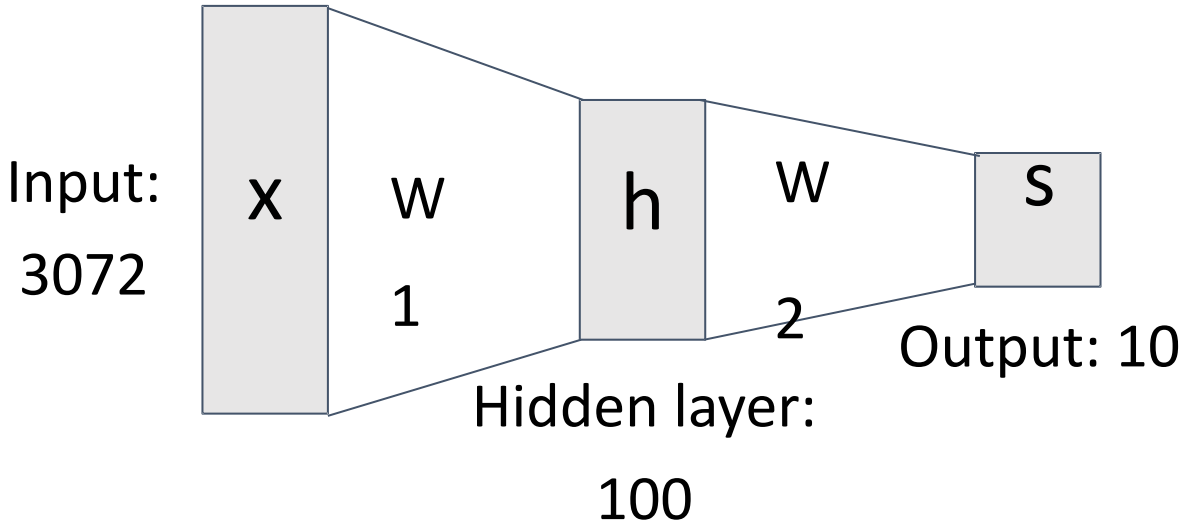
"Distributed representation":

Most templates not interpretable!



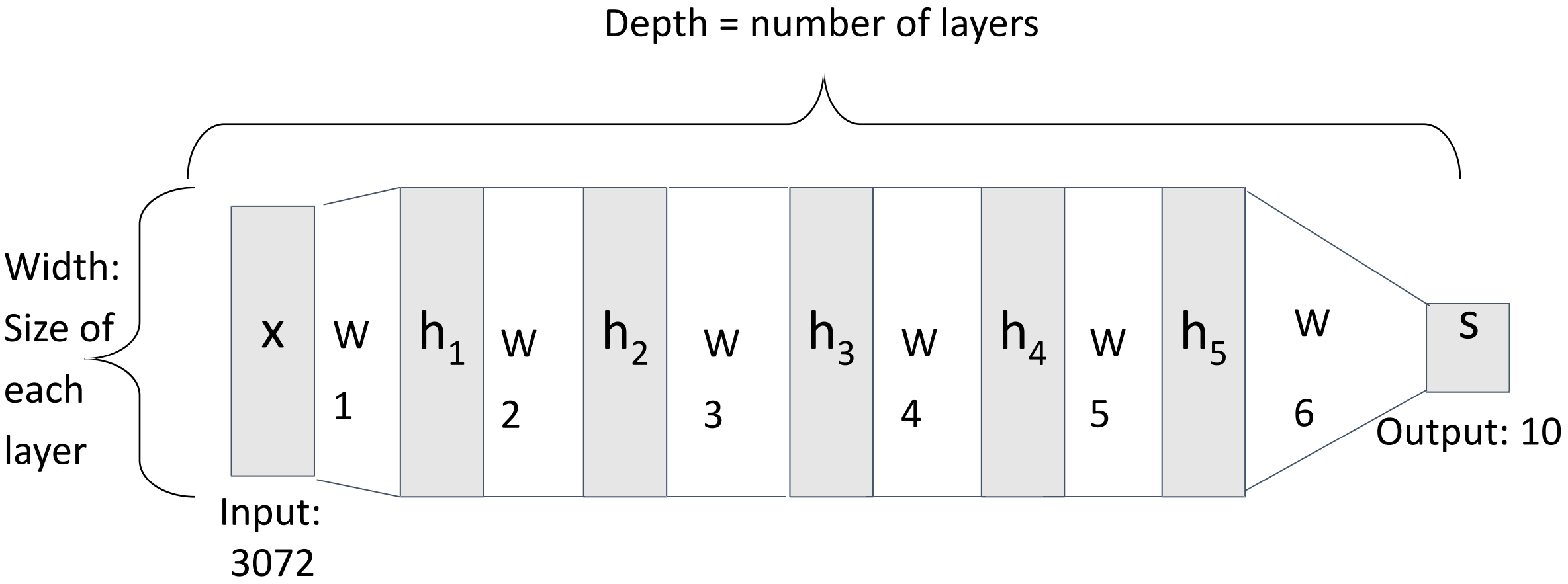
(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Deep Neural Networks



$$s = W_6 \max(0, W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

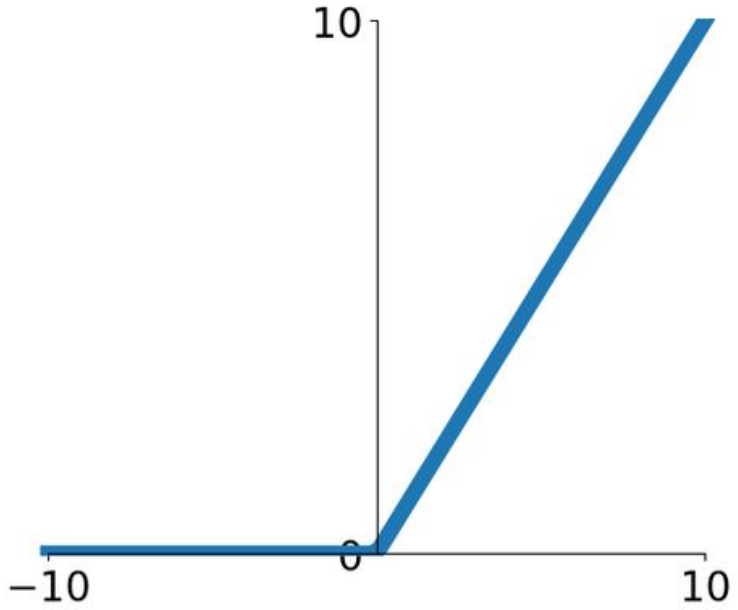
Activation Functions

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called “Rectified Linear Unit”

This is called the **activation function** of the neural network

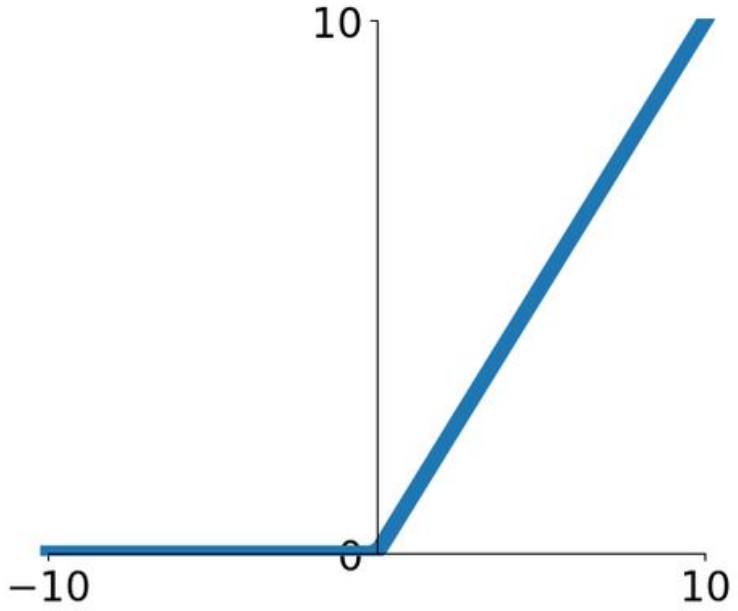


Activation Functions

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called “Rectified Linear Unit”



This is called the **activation function** of the neural network

Q: What happens if we build a neural network with no activation function?

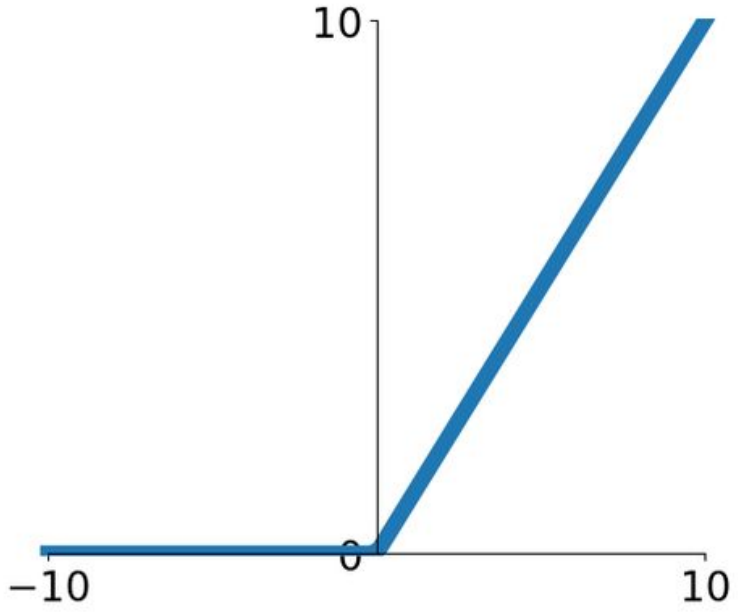
$$s = W_2 W_1 x$$

Activation Functions

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called “Rectified Linear Unit”



This is called the **activation function** of the neural network

Q: What happens if we build a neural network with no activation function?

$$s = W_2 W_1 x$$

$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H} \qquad s = W_3 x$$

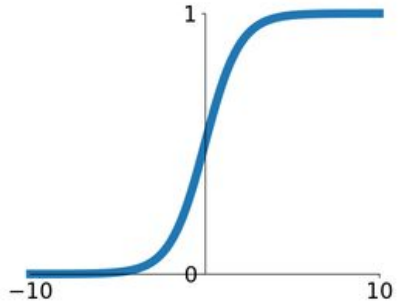
A: We end up with a linear classifier!

Activation Functions

ReLU is a strong default for hidden layers

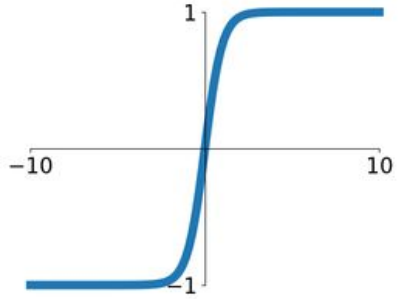
Sigmoid

$\sigma(x) = 1 / (1 + e^{-x})$



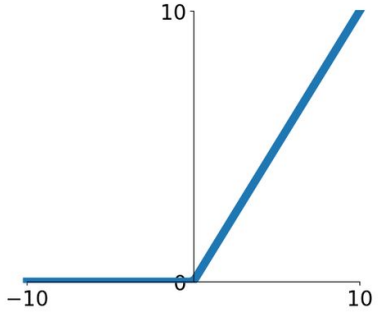
tanh

tanh(x)



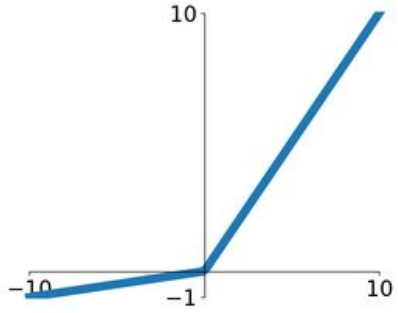
ReLU

max(0, x)



Leaky ReLU

max(0.1x, x)

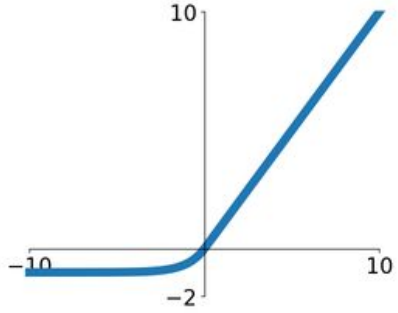


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

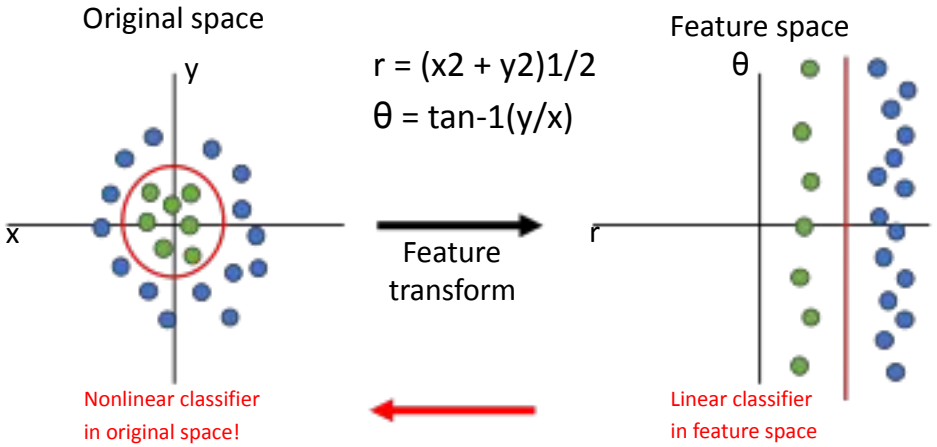
ELU

x if $x \geq 0$; $\alpha(e^x - 1)$ otherwise



Summary

Feature transform + Linear classifier allows nonlinear decision boundaries



Neural Networks as learnable feature transforms

